



6



13



1560 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Graph



د. مدحت الصوص

RB Informatics; 7/18/2025

محتوى مجاني غير مخصص للبيع التجاري

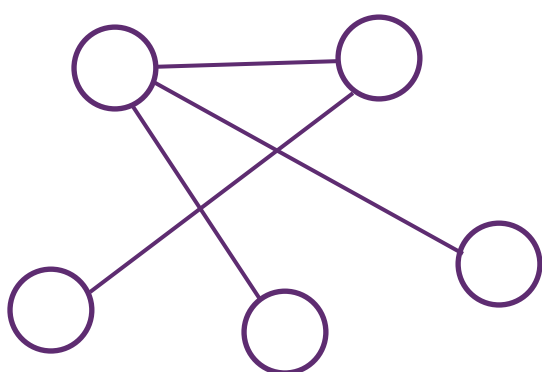
خوارزميات ٢

البيان Graph

تعلمنا في المحاضرات السابقة عن الأشجار، وأنها تتكون من عقد ووصلات مع شروط معينة حتى نطلق عليها اسم الشجرة.

- ولكن ماذا لو لم تتحقق هذه الشروط؟
- أي إذا كان لدينا مجموعة من العقد والوصلات بدون شروط لطريقة الوصل ما بين العقد.
- نطلق على هذه المجموعة Graph وهو الحالة العامة من الأشجار أي أن الأشجار هي حالة خاصة من ال Graph.
- لا يوجد عقدة أب أو عقدة ابن ب Graph بل تكون العقدة مجاورة لعقدة أخرى، أي إذا كان عقدتان موصولتان ببعضهما البعض نطلق عليهما عقدتان متجاورتان.

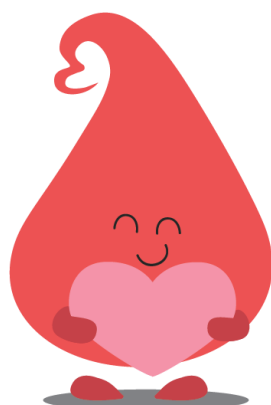
↪ شكل ال Graph:



$$G = \langle V, E \rangle$$

عدد العقد
Vertex

عدد الوصلات
EDGE



Type Of Graph

🔗 يقسم ال Graph إلى نوعين:

1. بيان موجه Directed Graph:



وهو أن تكون الوصلات موجهة باتجاه واحد من العقدة A إلى العقدة B. أي تكون الوصلة لها بداية ونهاية.

2. بيان غير موجه Undirected Graph:



تكون الوصلات بلا اتجاه معين وتذهب بكلتا الجهتين. أي لا يكون للوصلة بداية ونهاية.

🔗 كما يقسم ال Graph أيضاً إلى نوعين:

1. بيان موزون Weighted Graph:



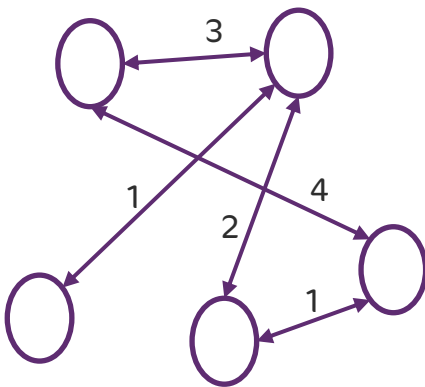
وهو أن يكون للوصلات ما بين العقد وزن أو قيمة معينة. مثلاً: هنا الوصلة ما بين A و B لها القيمة (5).

2. بيان غير موزون Unweighted Graph:

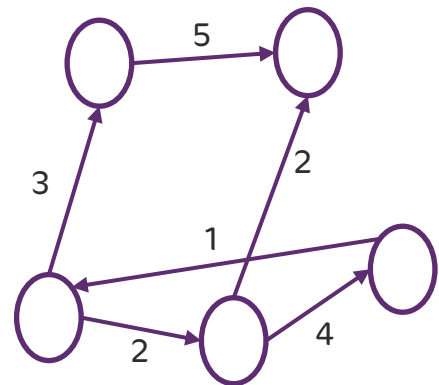


وهو ألا يكون للوصلات وزن أو قيمة. أي نصل فقط بين العقدتين.

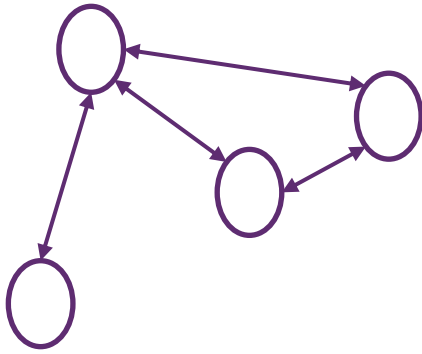
🔗 أصبح لدينا (4) أنواع لل Graph:



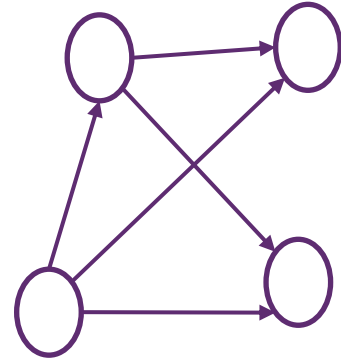
2. بيان غير موجه موزون



1. بيان موجه موزون



4. بيان غير موجه غير موزون



3. بيان موجه غير موزون

ملاحظة:

الأشجار هي بيان موجه غير موزون لأنها تحتوي على وصلات بدون أوزان وباتجاه واحد فقط.

أمثلة على ال Graph

- شبكة الطرق في مدينة: كل منطقة في المدينة تعبر عن عقدة والشوارع تعبر عن الوصلات بين هذه العقد.
- كل شارع يكون لديه مسافة معينة.
- كل شارع إما أن يكون باتجاه واحد أو باتجاهين متعاكسين.
- فإذا مثلنا الشبكة على Graph فسيكون Weighted Undirected Graph إذا كان الطريق باتجاهين، أو يكون Weighted Directed Graph إذا كان الطريق باتجاه واحد.

البيان البسيط Simple Graph

ليكون البيان بسيط، يجب أن يحقق الشرطين التاليين:

- لا يوجد وصلة تصل العقدة بذاتها.
- لا يوجد وصلتان بين نفس العقدتين على نفس الجهة.



ملاحظة:

- إذا كان يوجد وصلتان بين نفس العقدتين ولكن بعكس الجهة فإن ال Graph يبقى بسيطاً.



عدد الوصلات الأعظمي في ال Simple Graph

1. في البيان موجه Directed Graph:

نلاحظ في البيان الموجه أن كل عقدة تستطيع الاتصال بجميع العقد عدا العقدة نفسها، وبالتالي يكون عدد الوصلات الأعظمي هو:

$$m \leq n(n - 1)$$

حيث m هو عدد الوصلات و n هو عدد العقد.

2. في البيان غير الموجه Undirected Graph:

في البيان غير الموجه كل وصلة تكافئ وصلتين في البيان الموجه (لأن الوصلة تكون باتجاهين متعاكسين).



في البيان غير الموجه



في البيان الموجه

لذلك فإن عدد الوصلات الأعظمي في البيان غير الموجه يكون نصف عدد الوصلات الأعظمي في البيان الموجه.

$$m \leq \left(\frac{n(n - 1)}{2} \right)$$

البيان الجزئي Subgraph

هو مجموعة جزئية من عقد ووصلات ال Graph الأساسي، ولكن الوصلات الجزئية التي نأخذها للبيان الجزئي حصراً تكون تصل بين العقد الجزئية التي أخذناها.

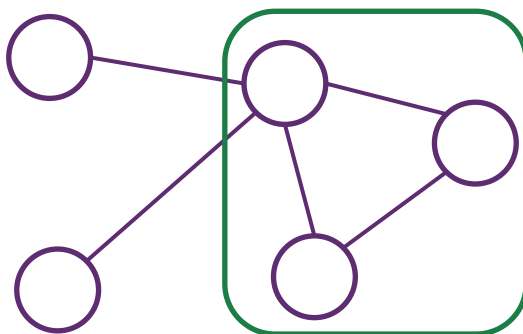
ليكن لدينا البيان $G = \langle V, E \rangle$ نقول أن البيان $G' = \langle V', E' \rangle$ هو بيان جزئي له إذا تحقق:

$$1. E' \subseteq E, V' \subseteq V$$

أي أن جميع العقد والوصلات يجب أن تنتمي للبيان الأساسي.

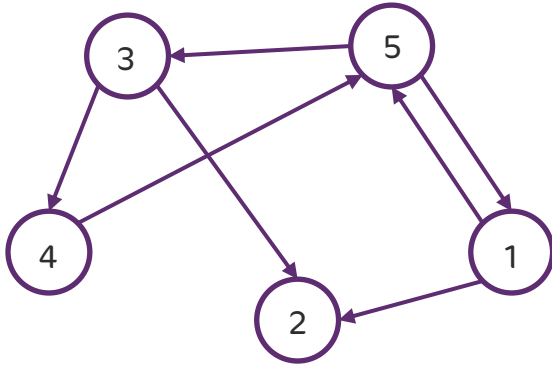
$$2. \forall (a, b) \in E' \Rightarrow a, b \in V'$$

أي أن كل وصلة في هذا البيان يجب أن ينتمي طرفاها إلى العقد الجزئية.



بيان جزئي

المسار Path In Graph



هو تتالي عدد من العقد، بحيث توجد وصلة بين كل عقدتين متتاليتين .
مثلاً في ال Graph المجاور واحد من المسارات هو:



مسار آخر:



المسار البسيط Simple Path

هو مسار لا تكرر فيه العقدة مرتين، في المثال السابق:



هذا مسار بسيط



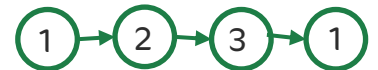
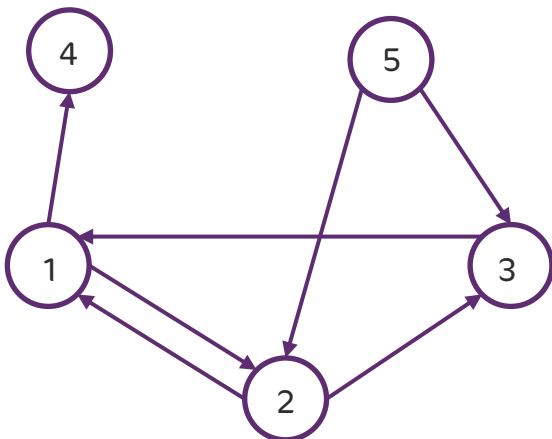
هذا ليس مسار بسيط

(بسبب تكرار العقد)



الحلقة Cycle

هو مسار يكون له نفس البداية والنهاية أي يبدأ من عقدة وينتهي عند نفس العقدة.
المسار الآتي:



بدأنا من ال (1) وانتهينا عنده لذلك نسمي

هذا المسار حلقة.

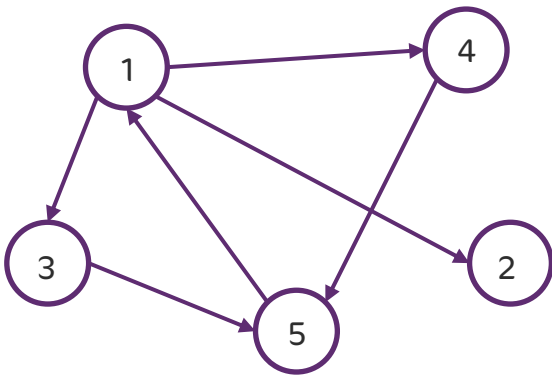


درجة العقدة $Deg(V)$

وهو عدد العقد المجاورة لهذه العقدة.

- $Deg^-(V)$: هو عدد الوصلات الواردة إلى هذه العقدة.
 - $Deg^+(V)$: هو عدد الوصلات الخارجة من هذه العقدة.
 - ودرجة العقدة بشكل عام هي مجموع الوصلات الواردة والخارجة.
- $$Deg(V) = Deg^-(V) + Deg^+(v)$$

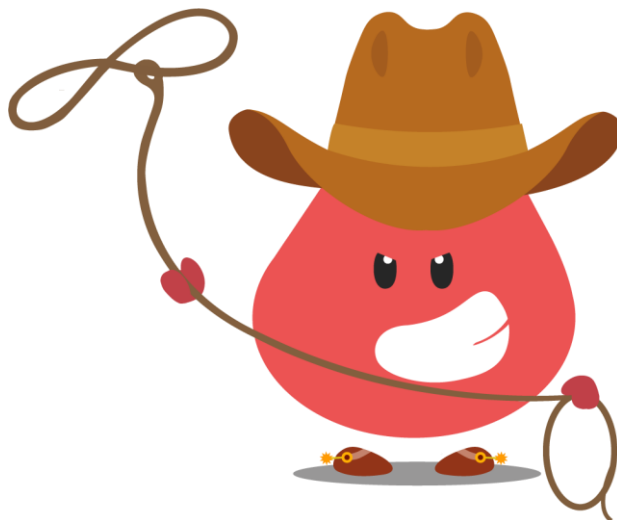
مثال:



- $Deg^+(1) = 3$
- $Deg^-(1) = 1$
- $Deg(1) = 1 + 3 = 4$

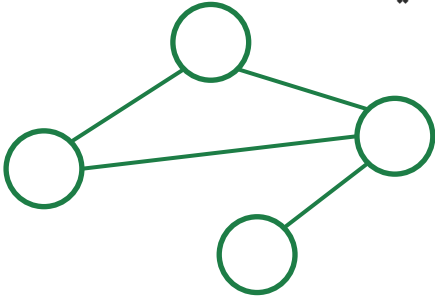
ملاحظة:

- في البيان الموجه:
 $Deg(V) = Deg^-(V) + Deg^+(V)$
- في البيان غير الموجه:
 $Deg(V) = Deg^-(V) = Deg^+(V)$

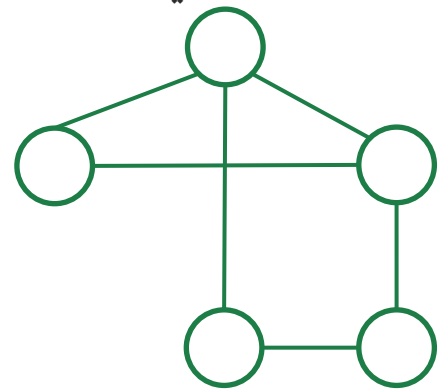


البيان المتصل بقوة Strongly Connected Graph

وهو أن يكون كل عقدة في ال Graph لديها مسار يصل بينها وبين أي عقدة أخرى في هذا ال Graph.



هذا ليس بيان متصل بقوة

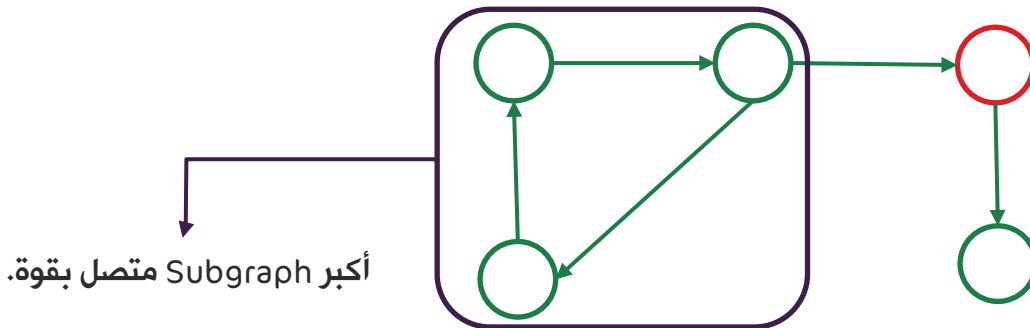


هذا بيان متصل بقوة



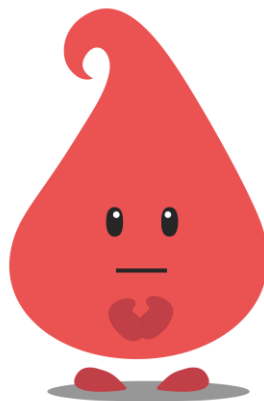
المركبة المتصلة بقوة Strongly Connected Component

وهو أكبر Subgraph متصل بقوة في ال Graph الأساسي.



ملاحظة:

- العقدة لا يمكنها الوصول إلى جميع العقد (لا يمكنها الوصول إلى العقدة التي قبلها) لأن ال Graph هو Directed Graph، ولذلك لا يمكن اعتبارها ضمن أكبر Subgraph متصل بقوة.



تخزين ال Graph

يوجد طريقتان لتخزين ال Graph:

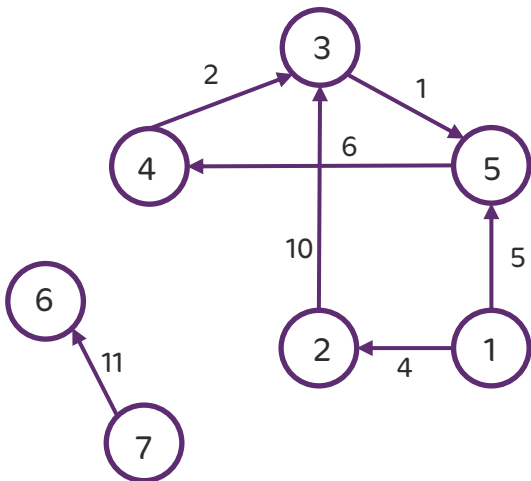
■ سلاسل التجاور

■ مصفوفة التجاور

1. مصفوفة التجاور:

- هي مصفوفة ثنائية حجمها هو $(n \times n)$ حيث n هو عدد العقد.
- كل عقدة تمتلك سطر وعمود.
- العقدة التي نبدأ منها الوصلة تكون في الأسطر.
- العقدة التي تنتهي فيها الوصلة تكون في الأعمدة.
- نخزن المسافات في نقطة تقاطع الأسطر مع الأعمدة للعقدة المرتبطة.

مثال: لنفرض أن لدينا هذا ال Graph:



- عدد العقد هو (7) لذلك سنشكل مصفوفة ثنائية مكونة من (7×7) عنصر.
- نبدأ من السطر الأول:
- العقدة (1) مرتبطة مع العقدة (2) بوصلة وزنها (4) لذلك عند السطر (1) في عمود العقدة المشار إليها في نهاية الوصلة (العقدة (2)) نضع الوزن.
- العقدة (2) مرتبطة مع العقدة (3) بوصلة وزنها (10) لذلك عند سطر العقدة (2) نضع وزن الوصلة في العمود الذي يحوي العقدة (3) وهو في حالتنا العمود الثالث.
- وهكذا نكمل مع باقي العقد.

	1	2	3	4	5	6	7
1		4			5		
2			10				
3					1		
4			2				
5				6			
6							
7						11	

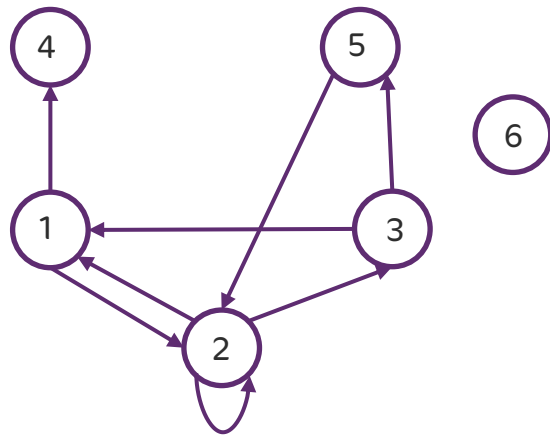


- في البيان الموزون نضع وزن الوصلة في مكان التقاطع ما بين عقدة البداية وعقدة النهاية.
- أما في البيان غير الموزون، فمن الممكن أن تكون المصفوفة الثنائية من النوع boolean ، فإذا كان لدينا وصلة ما بين عقدتين نضع القيمة true في مكان التقاطع ما بين عقدة البداية وعقدة النهاية لأننا فقط نريد معرفة العقد التي تصلها هذه الوصلة وإلا فتكون false.
- نستطيع الوصول لأي عنصر مباشرة في مصفوفة التجاور.
- حجم الذاكرة كبير (n^2) لأننا نحجز حجم n^2 دون الحاجة لاستخدامه (نحجز حجم للوصلات غير الموجودة أيضا).

2. سلاسل التجاور:

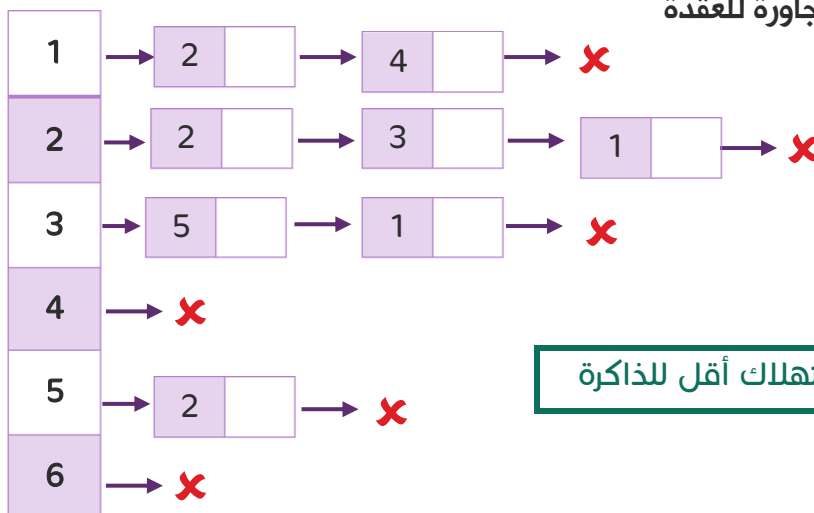
- نقوم بتعريف مصفوفة أحادية عناصرها هي Linked List ويكون عدد عناصر هذه المصفوفة بنفس عدد عقد ال Graph.
- في كل عنصر من المصفوفة (أي في كل Linked List) نخزن الوصلات والعقد المجاورة للعقدة الأساسية.

مثال: لنفرض أن لدينا هذا ال Graph:



- عدد العقد هو (6) لذلك نحجز مصفوفة أحادية عناصرها هي Linked List وعدد عناصرها هو (6).
- العقدة (2):
مجاوراتها: العقد (2) و (3) و (1).
وبالتالي ال Linked List تمثل العقد المجاورة للعقدة الأساسية (2).
- العقدة (1):
مجاوراتها: العقد (2) و (4).
وبالتالي ال Linked List تمثل العقد المجاورة للعقدة الأساسية (1).
وهكذا نكمل مع باقي العقد.

العقد الأساسية



نلاحظ استهلاك أقل للذاكرة

خوارزمية الفرز الطوبولوجي Topological Sort

هي خوارزمية ترتيب خطي للعقد في بيان موجه، حيث تتم طباعة العقدة بعد التأكد من طباعة جميع العقد المجاورة التي تشير إليها.

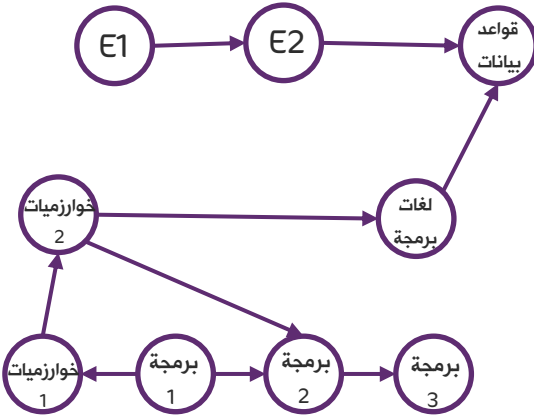
ملاحظة: يجب أن لا يحتوي البيان على حلقات، كما يجب أن يكون موجه Directed Graph.

مثال:

يوضح المثال التالي ترتيب بعض المقررات الجامعية.

بالنظر للبيان جانباً نلاحظ ما يلي:

- لا يمكن دراسة خوارزميات 2 قبل دراسة خوارزميات 1.
- لا يمكن دراسة قواعد البيانات قبل دراسة لغات البرمجة وخوارزميات 2.

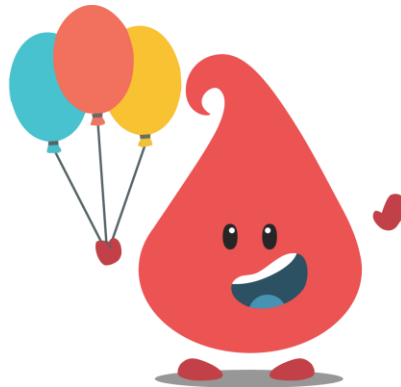


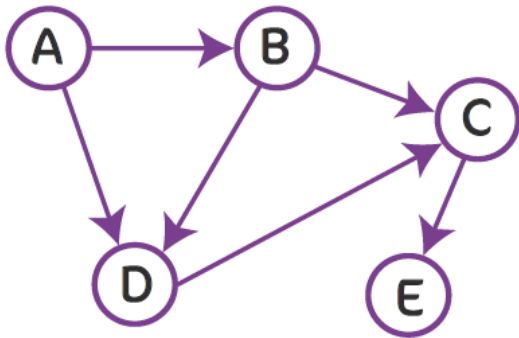
آلية تنفيذ الخوارزمية:

- في البداية سنحتاج لمصفوفة من أجل تخزين ترتيب العقد بالفرز الطوبولوجي.
- ثم سنخزن عدد الوصلات الواردة إلى كل عقدة في مصفوفة أحادية أخرى.
- سنحتاج أيضاً لمصفوفة زيارات [] visited من أجل التحقق في حال تمت زيارة كل عقدة.

الخطوات:

1. في البداية نأخذ العقدة التي تكون عدد الوصلات الواردة إليها هو 0 أو العقدة التي ليس لها آباء، أي أنه لا توجد أي عقدة تشير إليها ثم نقوم بطباعتها.
2. الآن نجعل هذه العقدة مزارة من خلال جعل القيمة المقابلة لهذه العقدة (-1) في مصفوفة وصلات العقد تجنباً لتكرار المرور على العقدة نفسها.
3. نزيل الوصلات الصادرة عن العقدة السابقة بأن ننقص عدد الوصلات للعقد المجاورة لها بمقدار 1، ثم نعاود أخذ العقد التي درجتها 0 أي $Deg^-(v) = 0$ ونكرر الخطوات السابقة حتى تصبح جميع قيم مصفوفة الوصلات (-1).





تطبيق مباشر:

$$Deg^-$$

0	1	2	2	1
A	B	C	D	E

>> A

$$Deg^-$$

-1	0	2	1	1
A	B	C	D	E

>> A B

$$Deg^-$$

-1	-1	1	0	1
A	B	C	D	E

>> A B D

$$Deg^-$$

-1	-1	0	-1	1
A	B	C	D	E

>> A B D C

$$Deg^-$$

-1	-1	-1	-1	0
A	B	C	D	E

>> A B D C E

$$Deg^-$$

-1	-1	-1	-1	-1
A	B	C	D	E

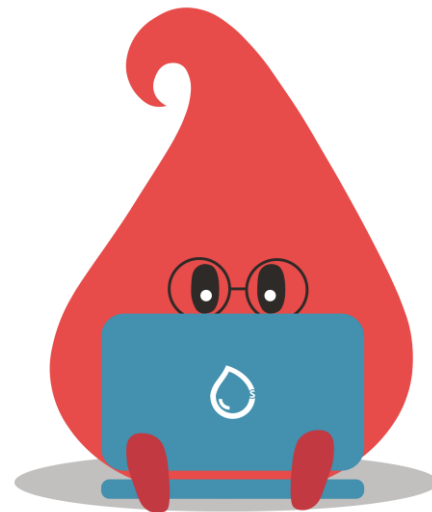
"Done"

■ نقوم بإنشاء المصفوفة.

■ نبحث عن الخانات التي تحوي (0) وسنطبع (A).

■ سنقوم بإسناد القيمة (-1) للخانات المقابلة للعقد التي تمت طباعتها ، وسنبحث عن العقد المجاورة الصادرة عن العقد التي تمت طباعتها ونقل عدد وصلاتها بمقدار (1).

■ سنعاود البحث عن قيم صفرية بالمصفوفة ونقوم بطباعتها وهكذا حتى تصبح جميع قيم المصفوفة Deg هي (-1).



"The only way to do great work is to love what you do."
- Steve Jobs

Implementation Of The Algorithm

```

void Topological_Sort ( bool Graph [ ] [ ]
) {
    int Deg [n] = 0 ;
    for ( i = 1 ; i < n ; i++ ) {
        for ( j = 1 ; j < n ; j++ ) {
            if ( Graph [i][j] == true ) {
                Deg [j] ++ ;
            }
        }
    }
    while (true) {
        int x = 1 ;
        while ( Deg [x] != 0 && x <= n ) {
            x++ ;
        }
        if ( x = n + 1 ) {
            break ;
        }
        print (x) ;
        Deg [x] = -1 ;
        for ( i = 1 ; i <= n ; i++ ) {
            if ( Graph [x][i] == true ) {
                Deg [i] -- ;
            }
        }
    }
}

```

■ عرفنا تابع *void* بوسيط هو البيان على شكل مصفوفة بوليانية ثنائية.

■ عرفنا مصفوفة *Deg* التي سنخزن فيها الوصلات الداخلة إلى كل عقدة.

■ قمنا بإنشاء حلقتين *for* متداخلتين من أجل معرفة ما إذا كان يوجد وصلة من *i* إلى *j* وفي حال وجودها سنخزن عددها في الخانة *j* من المصفوفة *Deg*.

■ ننشئ حلقة *while* شرط استمرارها محقق دائماً وفي داخلها ننشئ حلقة أخرى للبحث عن عقد لا تشير إليها أي وصلة.

- في حال العثور على عقدة لا تملك أي وصلات داخلة إليها سنقوم بطباعتها، ثم نسند قيمة (-1) للخانة المقابلة لها بمصفوفة ال *Deg* لمنع التكرار.

- ننشئ حلقة *for* للبحث عن العقد المجاورة التي لها وصلات من العقدة ذات الصفر وصلة، فإذا وجدناها سنقوم بإنقاص عدد الوصلات الداخلة إليها.

■ تستمر حلقة *while (true)* في العمل ويستمر البحث عن عقدة $Deg[x] == 0$ (أي لا يوجد وصلات واردة إليها)، وإذا لم يتم العثور على أي عقدة كهذه، فإن العداد x يستمر بالازدياد حتى يصل إلى $n + 1$ ، وهنا يتم كسر الحلقة (*break*) لأن جميع العقد إما تمت معالجتها أو لا يمكن معالجتها (في حال وجود *cycle*).

التعقيد الزمني:

- في البداية، يتم بناء مصفوفة $Deg[]$ باستخدام حلقتي for متداخلتين، كل منها يتكرر حتى n ، وبالتالي تعقيد الحلقتين هو: $O(n^2)$
- بعد ذلك، حلقة $while(true)$ يوجد بداخلها:
 1. حلقة $while$ للبحث عن العقد التي لها $Deg = 0$ والتي تعقيدها هو $O(n)$.
 2. حلقة for لإنقاص قيم المصفوفة $Deg[]$ والتي تعقيدها هو $O(n)$ أيضاً.
- نظراً لأن هذه العمليات تتكرر حتى تتم معالجة جميع العقد (أي n مرة)، فإن تعقيد حلقة $while true$ يصبح أيضاً: $O(n^2)$.

إذاً، التعقيد الكلي للخوارزمية هو:

$$O(n^2 + n^2) = O(n^2)$$

ملاحظة آريسيزية:

- في الفرز الطوبولوجي لا يمكن أن يوجد Cycles (حلقات) في ال graph، لأنه إن وجد حلقة لن نستطيع الخوارزمية أن تمر على جميع عناصر ال graph.
- لذلك نطلق على هذا ال graph : DAG (Directed Acycle Graph) وهو غراف لا يحتوي على حلقات بداخله.
- في حال كان البيان لا يمكن ترتيبه وفق الخوارزمية السابقة (بسبب احتواءه على حلقات)، فإن القيم النهائية للمصفوفة $Deg[]$ لن تكون جميعها (-1) ، حيث سنصل لمرحلة لا نستطيع أن نقرر فيها أي عقدة نأخذ، وذلك لأن جميع العقد المتبقية يوجد وصلات واردة إليها، وبسبب هذه الخاصية يمكن استعمال خوارزمية Topological Sort للتحقق فيما إذا كان البيان يحتوي على حلقات أم لا، وذلك عن طريق فحص قيم المصفوفة $Deg[]$ بعد تنفيذ الخوارزمية والتحقق فيما إذا كانت جميعها (-1) أم لا.

THE END ^_^